

CS5330: Pattern Recognition and Computer Vision

Project 4 Report: Calibration and Augmented Reality

Ahilesh Vadivel - 002055401

Description:

This project explores the fundamentals of camera calibration and augmented reality through a series of incrementally complex computer vision tasks implemented in C++ using OpenCV.

The project begins with the detection and sub-pixel refinement of checkerboard corners in a live video stream, establishing the foundation for all subsequent tasks. A camera calibration pipeline is then constructed, collecting corner observations across multiple frames and applying the Zhang calibration method via `cv::calibrateCamera` to recover the camera's intrinsic parameters, focal length, principal point, and lens distortion coefficients.

With a calibrated camera, `cv::solvePnP` is used to estimate the 6-DOF pose of the checkerboard target in real time, recovering the rotation and translation of the board relative to the camera coordinate system. This pose is then leveraged to project 3D points back into the image plane using `cv::projectPoints`, first to overlay coordinate axes and board corners, and then to render a fully constructed 3D virtual object, a rocket ship, that correctly tracks the board's motion and orientation.

Finally, the Harris corner detector is applied to a live video stream to investigate robust feature detection and its potential role in target-free augmented reality systems.

Task 1: Detect and Extract Target Corners

Target Used

This project uses the OpenCV checkerboard pattern as the calibration and AR target. When the long axis is oriented horizontally, the board presents 9 columns x 6 rows of internal corners, yielding 54 detectable corner points per frame. Each square is treated as a 1x1 unit in the coordinate system used for object placement.

The target was displayed on a screen/printed and mounted on a flat surface to minimize warping and ensure consistent planarity across frames.

Limitations and Challenges

Partial occlusion: If any portion of the board is outside the camera frame or obstructed, findChessboardCorners returns false and no corners are detected. The algorithm requires the complete pattern to be visible.

Illuminations: Glare from screen-displayed boards, shadows from overhead lights, or low-contrast conditions reduce detection reliability. Adaptive thresholding mitigates this but does not eliminate it.

Motion Blur: Fast camera movement causes blur that degrades corner localization. Sub-pixel accuracy cannot be achieved under significant blur.



Task 2: Select Calibration Image

Camera calibration estimates the intrinsic parameters of the camera, the focal lengths, principal point, and lens distortion coefficients. These parameters define the mapping between 3-D world coordinates and 2-D image coordinates and are required for all subsequent pose-estimation tasks.



Task 3: Calibrate the Camera



```
Camera matrix before calibration:
[1, 0, 320;
 0, 1, 240;
 0, 0, 1]

Camera matrix after calibration:
[481.897716773483, 0, 327.616030446833;
 0, 481.897716773483, 243.7465172280047;
 0, 0, 1]

Distortion coefficients:
[-0.04357525472191699;
 0.325938387753715;
 0.004261207531433346;
 0.0006355547839636326;
 -0.6402129745743215]

RMS reprojection error: 1.28335 px
```

The RMS reprojection error of 1.28 px is just above the ideal threshold of 1.0 px but well within the acceptable range for this project. The identical values of f_x and f_y (481.90) confirm square pixels, as expected with the CALIB_FIX_ASPECT_RATIO flag. The principal point (327.62, 243.75) is very close to the image center (320, 240). The higher k_2 and k_3 distortion coefficients suggest moderate radial distortion, which is typical for a webcam lens.

Task 4: Calculate Current Position of the Camera

```
Rotation vector:
[3.033648125655258;
-0.02076809211086484;
-0.08109832557746685]
Translation vector:
[-3.210601047018134;
-2.868414807604365;
25.27854312472677]

Rotation vector:
[2.983401127964358;
-0.08363083357487969;
-0.122118608504724]
Translation vector:
[-3.934464061484882;
-3.620930193860481;
25.59182036498462]

Rotation vector:
[2.88647550751149;
-0.2394952832873309;
-0.1954771214987906]
Translation vector:
[-5.915312784223064;
-4.02493322918145;
26.0498347136245]
```



Observations:

Center position: $tx = -5.7633$, $ty = -5.4469$, $tz = 27.4086$

Left position: $tx = -11.7242$, $ty = -4.8832$, $tz = 26.3072$

Right position: $tx = 1.0505$, $ty = -5.8707$, $tz = 25.5598$

Down position: $tx = -3.9524$, $ty = 0.6137$, $tz = 24.6419$

Up position: $tx = -4.7355$, $ty = -8.6066$, $tz = 24.2211$

Zoom in position: $tx = -3.9545$, $ty = -4.5073$, $tz = 11.4010$

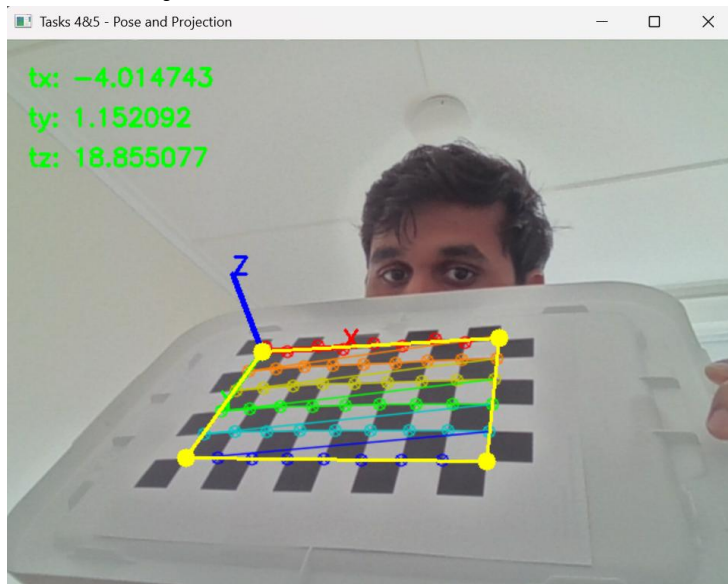
Zoom out position: $tx = -3.4128$, $ty = -2.7808$, $tz = 25.2677$

tx (left/right): Moving the board to the left caused tx to decrease from -5.76 to -11.72, and moving it to the right caused tx to increase to 1.05. This is consistent with the expected behavior, tx directly tracks horizontal displacement of the board relative to the camera.

ty (up/down): Moving the board upward caused ty to decrease from -5.45 to -8.61, and moving it downward caused ty to increase toward 0.61. The negative direction for upward movement is consistent with the coordinate system definition where Y is negative downward on the board.

tz (depth): Moving the board closer to the camera caused tz to drop significantly from 27.41 to 11.40, while moving it farther away kept tz near 25-27. This confirms that tz correctly represents the depth of the board from the camera in units of board squares.

Task 5: Project Outside Corners or 3D Axes

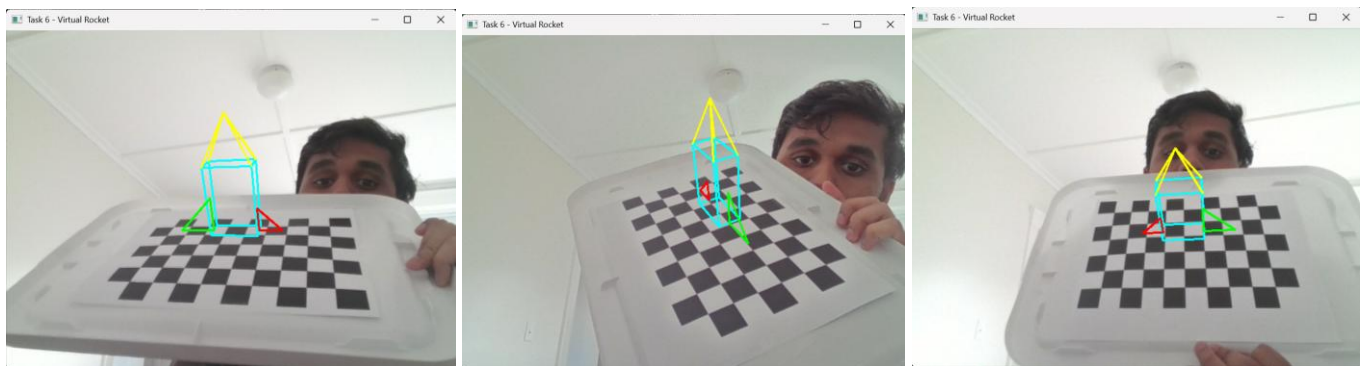


The projected outer corners correctly align with the physical corners of the checkerboard as the board is moved and rotated. The 3D axes remain anchored to the upper-left corner of the board and orient correctly as the board is tilted or rotated, confirming that the pose estimate and projection pipeline are working correctly end to end.

Task 6: Create a Virtual Object

The virtual object is a rocket ship floating above the center of the checkerboard. It is constructed entirely from 3D line segments projected into image space using `cv::projectPoints`. The rocket was chosen because it is asymmetric (the two fins have different sizes), which makes it easy to verify correct orientation as the camera moves.

The rocket is centered at approximately (4, -2.5) in board space, which places it near the middle of the 9x6 board. It floats above the board starting at $Z=1$ and extending to $Z=6$ (toward the viewer).



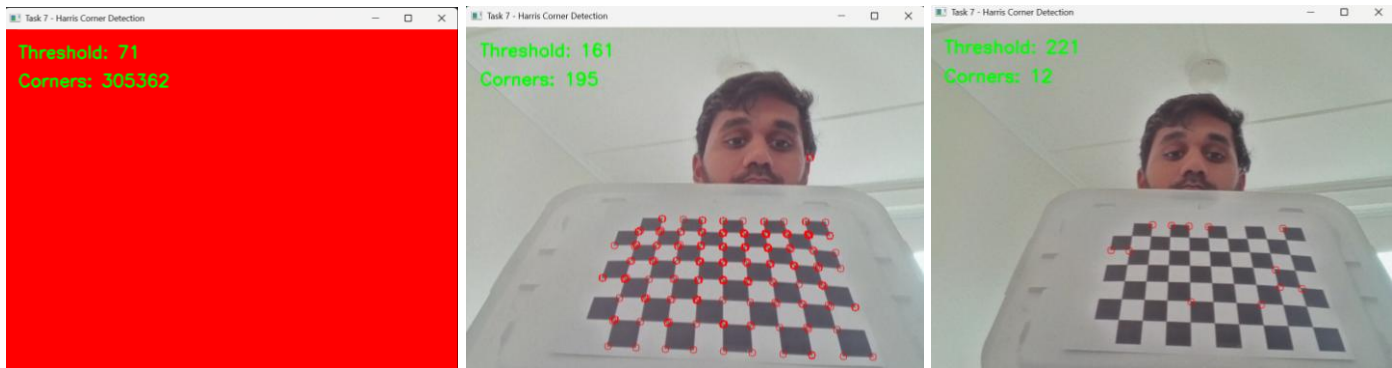
Video Link: <https://drive.google.com/file/d/15paevQa7xWUdkOUMczPvi3i6uTMQuVVb/view?usp=sharing>

The rocket correctly floats above the board and maintains its orientation as the camera is moved around the target. The asymmetric fins make it visually clear when the object is rotating correctly, the red and green fins appear on the correct sides as the board is tilted or rotated.

Task 7: Detect Robust Features

The threshold was adjusted interactively using '+' and '-' keys:

- Low threshold (< 100): A large number of weak corners are detected, including many false positives on textured surfaces and noise
- Medium threshold (approx. 150): A clean set of strong corners is detected, concentrated at clear geometric features
- High threshold (> 200): Only the strongest corners are detected; the checkerboard inner corners stand out clearly



Video Link: https://drive.google.com/file/d/1C2g4leBfwe5YUG7zRN_8gtqvrRnxPrA4/view?usp=sharing

Application to Augmented Reality:

Harris corners (or similar robust feature detectors) could serve as the basis for AR in several ways:

Target-free AR: Rather than requiring a known calibration target like a checkerboard, a system could detect Harris corners in the scene and match them across frames using descriptors (like SIFT or ORB). If enough corner correspondences are established between a reference image and a live frame, the homography or pose between them can be estimated using `cv::findHomography` or `solvePnP`, allowing a virtual object to be placed relative to any recognizable surface.

Robustness: Unlike the checkerboard, which requires the entire pattern to be visible, feature-based AR can work even when only part of the target is visible, since a subset of matched corners may be sufficient to estimate the pose.

Tracking: Harris corners can also be used with optical flow (`cv::calcOpticalFlowPyrLK`) to track feature points across frames, enabling smooth AR even when the target detection temporarily fails.

Reflection

This project provided valuable hands-on experience with the full pipeline of a camera calibration and augmented reality system, bridging theoretical concepts from the course with practical implementation challenges.

One of the most important takeaways was understanding how sensitive the calibration process is to the quality and variety of input frames. Early calibration attempts with insufficient angular variation produced reprojection errors above 2 pixels, highlighting how critical it is to sample a wide range of board orientations and positions. This reinforced the idea that good data collection is just as important as the algorithm itself.

Working with solvePnP deepened the understanding of how a camera's intrinsic parameters connect 3D world coordinates to 2D image space. Observing the rotation and translation vectors change in real time as the board was moved made the mathematics of the pinhole camera model tangible and intuitive in a way that lecture alone cannot fully convey.

Designing and projecting a 3D virtual object reinforced how all components of the pipeline, calibration, pose estimation, and projection, must work correctly together. A small error in any one stage propagates visibly into the final AR result.

Finally, experimenting with the Harris corner detector provided insight into how feature-based methods could extend AR beyond the constraints of a known calibration target, pointing toward more flexible and robust real-world applications.

Acknowledgements

Textbooks

- Maxwell, B. A. *Fundamentals of Computer Vision*. 2022. Referenced for core concepts in image formation, camera models, and feature detection.
- Shapiro, L. and Stockman, G. *Computer Vision*. Prentice-Hall, 2001. Referenced for foundational pattern recognition and computer vision theory.

OpenCV Documentation

- OpenCV. *Camera Calibration and 3D Reconstruction*. Referenced for the usage of findChessboardCorners, cornerSubPix, calibrateCamera, solvePnP, projectPoints, and cornerHarris.

AI Assistance

- Claude (Anthropic) used as a programming assistant to support code structure, debugging, and report writing throughout the project.